6부 전반 — 글자에서 의미로

앱 페이지 15~18 · 학생용과 동일 구조 + 정답·해설

> **핵심 메시지** "신경망에 들어가는 모든 것은 결국 벡터다. 글자도 예외가 아니다." 글자 → 코드포인트 → 토큰 → 임베딩 → 의미 공간으로 이어지는 사다리를 학생이 한 호흡으로 따라갈 수 있게 한다.

STEP 1. 텍스트가 숫자가 되기까지

1-1 "안녕하세요" — 정답

정답 글자 수 = **5**개, 총 UTF-8 바이트 = **5** × 3 = **15** 바이트.

한글 음절이 모두 3바이트인 이유 한글 완성형 음절(가~힣) 영역은 유니코드 U+AC00 ~ U+D7A3 (코드포인트 약 44032~55203 범위). 이 범위의 코드포인트는 UTF-8 인코딩 규칙상 모두 정확히 3바이트로 펼쳐진다.

1-2 한·영·이모지 — 정답

문자열	글자 수	UTF-8 바이트
"AI"	2	2 (각 1바이트)
"인공지능"	4	12 (각 3바이트)
"🤖🧠"	2	8 (각 4바이트)

이유 UTF-8은 ASCII 호환을 위해 자주 쓰는 영어를 1바이트로 짧게 둠. 한글·이모지처럼 코드포인트가 큰 글자는 더 많은 바이트가 필요. → "표현력과 호환성의 타협"이 만든 결과.

수업 팁 "바이트 수가 다르면 입력 길이가 다르고, 그게 모델 비용에 그대로 영향을 준다"는 점이 다음 STEP의 토큰 단가 이야기와 자연스럽게 연결됨.

STEP 2. 토큰 (Phase 16)

2-1 토큰 수 — 전형 결과 (GPT 계열 토크나이저 기준)

예문	글자 수	토큰 수
Hello world	11	2
Tokenization is fun	19	4
안녕하세요	5	5~7
공부했어요	5	5~8
GPT는 한글을 잘게 쪼개요	15	15~20

수업 팁 학생이 앱에서 보는 정확한 수치를 그대로 적게 한다(토크나이저 버전에 따라 약간씩 다름). 핵심 메시지: **한글 글자수 ≪ 토큰수**.

2-2 BPE 미니 — 예시 정답

예시 진행 (low/lower/newest/widest 코퍼스 기준 — 정확한 빈도는 앱 따라 조금씩 다름)
1단계: 가장 자주 붙는 페어 `e + s` → `es`
2단계: `es + t` → `est`
3단계: `l + o` → `lo` (또는 `w + e` → we)

메시지 "글자 → 자주 같이 등장하는 두 조각을 한 토큰으로 합쳐 어휘를 키운다." 이 절차의 반복이 BPE.

2-3 한글 토큰이 많아지는 이유 — 모범 답안

예시 학습 코퍼스에 한글 텍스트가 영어 텍스트보다 훨씬 적어서, 한글 음절·바이트 페어가 자주 등장하는 단위로 학습되지 못함 → 합쳐지지 못한 채 글자 단위 또는 바이트 단위로 남음 → 결과적으로 토큰 수↑.

자주 틀림 "한글이 더 복잡해서"라는 답. **복잡성이 아니라 학습 데이터의 빈도**가 핵심임을 짚어줄 것. 영어도 새로 만든 단어("metaverse" 같은)는 자주 잘게 쪼개진다.

STEP 3. 원-핫 → 임베딩 (Phase 17)

3-1 원-핫 + 코사인 — 정답

계산 "고양이" = (`1, 0, 0, 0, 0, 0`) (1번째 자리만 1로 가정)
"강아지" = (`0, 1, 0, 0, 0, 0`) (2번째 자리만 1)
내적 = 0, 크기 = 1·1 → cos = `0`

왜 문제인가 의미가 매우 비슷한 "고양이/강아지"와 의미가 전혀 다른 "고양이/컴퓨터"가 똑같이 cos = 0이 된다. 신경망이 "비슷한 단어를 비슷하게 다루는" 능력을 처음부터 갖지 못한다.

3-2 임베딩 그룹 — 모범 답안

예시 가까이 모이는 그룹: **{고양이, 강아지, 호랑이}**(동물), **{사과, 바나나}**(과일). 멀리 떨어진 단어: **컴퓨터**. 의미 카테고리가 같으면 자연스럽게 한 영역에 모이도록 학습이 진행된다.

3-3 임베딩 행렬 W — 정답

계산 W의 크기 = **6 × 3** = 학습할 숫자 **18**개

곱셈의 의미 원-핫 × W = W의 **해당 단어 행**을 그대로 꺼내오는 것과 같다. 즉 "임베딩 = W에서 그 단어 인덱스의 행 벡터"이다. 그래서 실제 구현은 행렬곱이 아니라 **인덱스 룩업**으로 한다.

수업 팁 "행렬곱이지만 사실은 룩업"이라는 트릭이 거대 모델에서도 그대로 쓰인다는 점을 짚으면, 학생들이 거대해 보이는 LLM도 익숙한 도구로 만들어졌다는 감각을 얻음.

STEP 4. Word2Vec (Phase 18)

4-1 의미 산수 — 전형 결과

예시 제대로 학습됐다면 4문제 모두 1순위로 기대 답이 등장. 학습량이 부족하면 1순위가 어긋나도 상위 3개 안에는 들어오는 경우 많음.

앱 운영 학습 step 1500이 기본. 학생들이 결과가 이상하게 나온다고 보고하면 step 3000으로 올려 재학습 시킬 것. seed에 따라 결과가 달라지므로, "정답 시드"가 있다는 점도 안내.

4-2 의미 차 = 벡터 차 — 정답

계산 z_1: 0.8 − 0.5 + 0.5 = **0.8**
z_2: 0.6 − 0.4 + 0.8 = **1.0**
결과 (0.8, 1.0) = vec(queen). **같다**.

두 화살표 정답 queen − king = (0.8−0.8, 1.0−0.6) = `(0, 0.4)`
woman − man = (0.5−0.5, 0.8−0.4) = `(0, 0.4)`
두 화살표는 **같다** (방향·크기 모두 동일).

의미 "king → queen"과 "man → woman"이 임베딩 공간에서 **같은 방향·같은 크기의 평행이동**이라는 뜻. 즉 두 변환이 모두 "남성 → 여성"이라는 동일한 의미 차이를 표현한다. 이것이 "**의미 차이 = 벡터 차**"라는 발상의 본질이고, LLM의 의미 처리의 출발점.

자주 틀림 두 벡터가 시작점이 다르니 다르다고 답하는 경우. **벡터(화살표)는 시작점이 아니라 방향·크기로 정의됨**을 짚을 것.

4-3 실패 사례 — 모범 답안

예시 학습 코퍼스에 등장 빈도가 낮은 단어("호랑이" 등)는 의미 산수 결과가 어긋나는 경우가 많다. 이유: 그 단어의 위치가 학습으로 잘 다듬어지지 못했기 때문.

교육적 의미 "임베딩의 품질은 코퍼스의 양과 다양성에 비례한다"는 데이터 시대의 격언을 학생이 직접 발견하게 됨.

STEP 5. 마무리 — 정답

정리 한 줄 정리 — 정답

글자는 (코드포인트)와 (UTF-8)를 거쳐 0과 1이 되고,
문장은 (토큰) 단위로 쪼개진 뒤 (임베딩) 벡터로 변환된다.
의미가 비슷한 단어가 (가까운) 곳에 모이도록 학습된 결과,
벡터의 (덧셈)과 (뺄셈)이 의미의 연산이 된다.

관찰·수행 평가 루브릭

수준	인코딩 (P15)	토큰 (P16)	임베딩 (P17)	Word2Vec (P18)
A 도달	코드포인트·UTF-8 차이를 정확히 설명. 한·영·이모지 바이트 수 차이의 이유를 안다.	한·영 토큰 수 차이의 원인을 학습 빈도로 설명. BPE 절차를 따라간다.	원-핫의 두 가지 한계를 정확히 설명. W의 크기와 룩업의 의미를 안다.	의미 산수 4문제 중 3 이상 적중. "의미 차 = 벡터 차"를 자기 말로 설명.
B 진행	UTF-8 바이트 수 비교는 하지만 이유는 모호.	한·영 차이는 알지만 BPE 절차를 못 따라감.	원-핫과 임베딩의 차이는 알지만 cos 계산 못함.	실험은 했으나 화살표 평행성을 못 짚음.
C 보충	"글자도 결국 숫자"라는 메시지를 흡수 못함.	토큰 수가 무엇을 세는지 모름.	원-핫이 무엇인지 모름.	의미 산수의 의의를 잡지 못함.

전이 포인트 여기까지가 "단어 한 개를 어떻게 표현하는가"였다. 6부 후반에서는 단어가 **줄지어 등장하는 시퀀스**를 다룬다. "어떤 단어에 집중할지"가 학습되는 어텐션이 등장하고, 그것이 트랜스포머와 GPT의 심장임을 알게 된다.